MediNexus

MVP — Intelligent Slot Booking & Unified Patient Health Passport

---

MVP Technical Plan — Version 1.0

March 2026

Lead Developer	Riyansh Verma
Program	B.Tech Computer Science, Semester 7
Stack	React (Vite) | FastAPI | PostgreSQL (Supabase)
MVP Scope	Slot Booking Engine + Patient Health Passport
Status	In Development

Working draft. Features subject to revision during development.

Contents

1 MVP Overview **2**
1.1 MVP Entities 2
1.2 Out of Scope for MVP 2

2 Entity 1 — Hospital Admin **2**
2.1 Hospital Registration & Onboarding 2
2.2 Doctor Roster Management 2
2.3 Service Catalogue Management 3
2.4 Slot Generation for Doctors 3
2.5 Appointment Dashboard 4

3 Entity 2 — Doctor **4**
3.1 Authentication & Profile Setup 4
3.2 Schedule & Availability Management 4
3.3 Daily Appointment Queue 4
3.4 Prescription Builder (Integrated CuraRx Module) 4
3.5 Patient Health Passport Access 5

4 Entity 3 — Patient **5**
4.1 Authentication & Profile Setup 5
4.2 Hospital & Doctor Discovery 5
4.3 Slot Booking with Soft-Lock 6
4.4 Real-Time Slot Availability via PostgreSQL LISTEN/NOTIFY and SSE 7
4.5 Cancellation & Waitlist Auto-Fill 7
4.6 Patient Health Passport 8
4.7 Cross-Hospital Record Sharing 8

5 PostgreSQL as Full Infrastructure **9**

6 FastAPI Layer Responsibilities **10**

7 Database Schema **10**

8 Technology Stack **11**

9 MVP Build Order **11**

10 Three-Portal Architecture **12**

MVP Overview

MediNexus MVP focuses on two core pillars of the larger platform:

Pillar 1 — Smart Slot Booking Engine
Real-time appointment booking with soft-lock overbooking prevention, automated waitlist filling on cancellation, and live slot availability broadcast to all connected clients.

Pillar 2 — Unified Patient Health Passport
A patient-controlled, persistent medical record that aggregates prescriptions, reports, and history across every hospital and clinic on the platform. Cross-hospital record sharing with explicit patient consent and time-limited access grants.

MVP Entities

Three entities are in scope for the MVP:

- **Hospital Admin** — Onboards the hospital, manages the doctor roster, configures the service catalogue, and generates appointment slots
- **Doctor** — Manages their schedule, runs appointment queues, issues digital prescriptions, and views shared patient records
- **Patient** — Discovers hospitals, books appointments, manages their health passport, and controls cross-hospital record sharing

Out of Scope for MVP

The following features are explicitly deferred to V2: AI chatbot, walk-in queue management, no-show ML prediction, multilingual UI, telemedicine, pharmacy integration, drug interaction checker, ABDM/ABHA integration, and super-admin UI.

Entity 1 — Hospital Admin

Hospital Registration & Onboarding

The hospital submits a registration form: name, type (hospital / clinic / solo practitioner), address, city, state, registration number, and primary admin contact. This creates a row in the `hospitals` table with `is_approved = false`. A platform admin toggles approval manually (no separate UI for MVP). On approval, the Hospital Admin receives credentials via Supabase invitation email.

Implementation: Supabase Auth creates the admin user. RLS policy: Hospital Admin can only read and write their own hospital row by matching `admin_id = auth.uid()`.

Doctor Roster Management

Hospital Admin adds doctors: name, specialisation, department, consultation fee, and default slot duration in minutes. Each doctor receives an invitation email via Supabase.

On acceptance, the doctor's `doctors` row is linked to the hospital via `hospital_id`.

Implementation: FastAPI endpoint `POST /hospitals/{id}/doctors/invite` creates a pending doctor record and triggers the Supabase invitation. A post-signup trigger links `doctors.user_id = auth.uid()` when the doctor completes signup.

Service Catalogue Management

Hospital Admin defines available services. MVP scope is limited to **doctor consultations** only. Each entry in `hospital_services` contains: service name, department, linked doctor, default duration in minutes, fee, and a pay-at-counter boolean.

PostgreSQL full-text search (`tsvector`) is added on `service_name` and `department` so patients can search "cardiologist" or "fever" and find relevant services.

Slot Generation for Doctors

Hospital Admin configures a doctor's availability: working days, start time, end time, and slot duration. The system generates individual slot rows for the next 30 days in the `appointment_slots` table. A PostgreSQL function handles this in a single database call:

```
CREATE OR REPLACE FUNCTION generate_doctor_slots(
  p_doctor_id UUID, p_start_time TIME,
  p_end_time TIME, p_duration_mins INT,
  p_working_days INT[], p_days_ahead INT
) RETURNS void AS $$
DECLARE
  curr_date DATE := CURRENT_DATE;
  slot_time TIMESTAMPTZ;
BEGIN
  FOR i IN 0..p_days_ahead LOOP
    IF EXTRACT(DOW FROM curr_date + i) = ANY(p_working_days) THEN
      slot_time := (curr_date + i + p_start_time);
      WHILE slot_time < (curr_date + i + p_end_time) LOOP
        INSERT INTO appointment_slots
          (id, doctor_id, slot_start, slot_end, status)
        VALUES
          (gen_random_uuid(), p_doctor_id, slot_time,
           slot_time + (p_duration_mins || '␣minutes')::interval,
           'available')
        ON CONFLICT (doctor_id, slot_start) DO NOTHING;
        slot_time := slot_time +
          (p_duration_mins || '␣minutes')::interval;
      END LOOP;
    END IF;
  END LOOP;
END;
$$ LANGUAGE plpgsql;
```

The `UNIQUE(doctor_id, slot_start)` constraint on `appointment_slots` guarantees no duplicate slots even if the function is called multiple times.

Appointment Dashboard

A read-only operational view showing today's appointment count per doctor, upcoming appointments for the week, and cancelled/no-show records. FastAPI endpoint: `GET /hospitals/{id}/appointments?date=today` with paginated response.

Entity 2 — Doctor

Authentication & Profile Setup

Doctor accepts an invitation email, sets password via Supabase Auth, and completes profile: bio, consultation fee confirmation, and prescription template preference (default or custom with clinic header).

Schedule & Availability Management

Doctor views generated slots in a calendar view and can:

- **Block individual slots** — sets `status = 'blocked'` via `PATCH /slots/{id}`
- **Mark a day as leave** — bulk update all slots on a date to `blocked` with a single SQL query; all booked patients are notified via Supabase Email + SMS through a FastAPI `BackgroundTask`

```
-- Bulk block all available slots on leave date
UPDATE appointment_slots
SET status = 'blocked'
WHERE doctor_id = $doctor_id
  AND slot_start::date = $leave_date
  AND status = 'available';
```

Daily Appointment Queue

Doctor's main working view. All booked appointments for today in chronological order. Each card shows: patient name, age, appointment type (new / follow-up), and a "View Health Passport" button if the patient has granted access. Doctor updates status: `waiting` → `in_consultation` → `completed` or `no_show`. Marking `no_show` increments `no_show_count` on the patient record for future use.

Prescription Builder (Integrated CuraRx Module)

Activated on completing a consultation. Full flow:

1. **Symptom Search** — PostgreSQL full-text search on the medicines table returns top 25 ranked results
2. **Medicine Name Search** — Direct lookup by brand or generic name
3. **Drag and Drop** — Medicines dropped into prescription panel
4. **Dosage Fields** — Per-medicine: dosage, frequency, duration, comment
5. **Output** — PDF generated by FastAPI (`weasyprint`), stored in Supabase Storage, emailed and SMS'd to patient

6. Prescription row is created and linked to `appointment_id` and `patient_id`; automatically appears in patient's Health Passport

PostgreSQL FTS on medicines:

```
-- Computed stored tsvector column
ALTER TABLE medicines ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (
    to_tsvector('english',
      coalesce(medicine_name, '') || '␣' ||
      coalesce(medicine_uses::text, '') || '␣' ||
      coalesce(description, '') || '␣' ||
      coalesce(therapeutic_class, ''))
  ) STORED;

CREATE INDEX medicines_fts_idx ON medicines USING GIN(search_vector);

-- Ranked symptom search (top 25)
SELECT id, medicine_name, composition, therapeutic_class,
       side_effects, substitutes, uses,
       ts_rank(search_vector, query) AS rank
FROM medicines,
     websearch_to_tsquery('english', $1) AS query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 25;
```

Patient Health Passport Access

FastAPI `GET /patients/{id}/passport` checks the `record_access_grants` table for a valid grant where `granted_to_doctor_id = auth.uid()` and `valid_until > now()`. If valid, returns aggregated prescription and report history. RLS at the database level enforces this independently of application logic.

Entity 3 — Patient

Authentication & Profile Setup

Sign up via Email + Password, Google OAuth, or Phone + OTP through Supabase Auth. Patient completes: full name, date of birth, blood group, known allergies (free text), and language preference. A `patients` row is created linked to `auth.uid()`.

Hospital & Doctor Discovery

Patient searches by hospital name, city, or speciality. FastAPI queries a PostgreSQL materialized view combining hospital and service data:

```
CREATE MATERIALIZED VIEW hospital_search_view AS
SELECT
  h.id AS hospital_id, h.name, h.city, h.address, h.type,
  to_tsvector('english',
    h.name || '␣' || h.city || '␣' || h.state || '␣' ||
    string_agg(hs.service_name || '␣' ||
      coalesce(hs.department,''), '␣')
```

```
  ) AS search_vector
FROM hospitals h
LEFT JOIN hospital_services hs ON hs.hospital_id = h.id
WHERE h.is_approved = true
GROUP BY h.id;

CREATE INDEX hospital_search_idx
  ON hospital_search_view USING GIN(search_vector);
```

The view is refreshed via `REFRESH MATERIALIZED VIEW CONCURRENTLY` called by FastAPI whenever a hospital or service record is updated. Results include a real-time "slots available today" boolean computed by checking for any `available` slot with `slot_start::date = CURRENT_DATE` for that hospital's doctors.

Slot Booking with Soft-Lock

This is the most critical flow in the MVP.

> **Booking Flow:**
> Patient selects doctor → views available slots → clicks a slot → **3-minute soft lock activates** → patient confirms → slot status becomes `booked` → confirmation sent via email + SMS

Step 1 — Acquire Soft Lock (atomic):

```
BEGIN;
UPDATE appointment_slots
SET
  status = 'soft_locked',
  locked_by = $patient_id,
  locked_until = now() + interval '3 minutes'
WHERE id = $slot_id
  AND status = 'available'  -- only if truly available
RETURNING id, slot_start, slot_end, doctor_id;
COMMIT;
-- 0 rows returned: slot taken. Return 409 Conflict.
-- 1 row returned: lock acquired. Return slot + countdown.
```

Step 2 — Confirm Booking (within 3 minutes):

```
BEGIN;
-- Re-verify lock still held by this patient
SELECT id FROM appointment_slots
WHERE id = $slot_id
  AND status = 'soft_locked'
  AND locked_by = $patient_id
  AND locked_until > now()
FOR UPDATE;
-- No row: lock expired. Return 410 Gone. Patient must restart.

INSERT INTO appointments
  (id, slot_id, patient_id, doctor_id, hospital_id,
   service_id, status)
VALUES
  (gen_random_uuid(), $slot_id, $patient_id, $doctor_id,
```

```
   $hospital_id, $service_id, 'booked');

UPDATE appointment_slots
SET status = 'booked', locked_by = NULL, locked_until = NULL
WHERE id = $slot_id;
COMMIT;
```

Lock Expiry Cleanup — A `pg_cron` job (or FastAPI `APScheduler` fallback) runs every 60 seconds:

```
UPDATE appointment_slots
SET status = 'available', locked_by = NULL, locked_until = NULL
WHERE status = 'soft_locked'
  AND locked_until < now();
```

The React frontend receives `locked_until` from FastAPI and renders a countdown timer. If the timer expires before confirmation, the UI shows "Slot no longer available" and calls `PATCH /slots/{id}/release` to free the lock early.

Real-Time Slot Availability via PostgreSQL LISTEN/NOTIFY and SSE

When a patient browses slots, availability updates live without page refresh. A PostgreSQL trigger fires on every slot status change:

```
CREATE OR REPLACE FUNCTION notify_slot_change()
RETURNS trigger AS $$
BEGIN
  PERFORM pg_notify('slot_updates',
    json_build_object(
      'slot_id',    NEW.id,
      'doctor_id',  NEW.doctor_id,
      'status',     NEW.status,
      'slot_start', NEW.slot_start
    )::text);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER on_slot_status_change
AFTER UPDATE OF status ON appointment_slots
FOR EACH ROW EXECUTE FUNCTION notify_slot_change();
```

FastAPI maintains a persistent `asyncpg` connection listening on `slot_updates` and pushes changes to React clients via **Server-Sent Events (SSE)**. React subscribes using `EventSource` and updates the slot grid in real time when any slot's status changes. A `: keepalive` comment is sent every 30 seconds to prevent connection timeout.

Cancellation & Waitlist Auto-Fill

Patient cancels from their dashboard up to a configurable threshold before the appointment time (e.g., 2 hours). On cancellation:

1. Appointment status set to `cancelled`
2. Slot status released to `available`

3. FastAPI queries the waitlist for the first patient with `status = 'waiting'`, ordered by `queued_at ASC`
4. That patient's waitlist entry is updated to `status = 'offered'` with `offer_expires_at = now() + interval '10 minutes'`
5. Supabase Email + SMS sent with a one-click acceptance deep link
6. If not accepted in 10 minutes, a background task marks it `expired` and cascades to the next patient in the queue

Patient Health Passport

A tabbed interface with three sections:

- **Prescriptions** — All prescriptions from all doctors on the platform, sorted newest first. Each entry shows doctor name, hospital, date, and medicines. Full prescription downloadable as PDF via signed Supabase Storage URL.
- **Reports** — All lab and imaging reports uploaded by hospital staff. Shows report name, type, hospital, date, and download link.
- **Profile & Conditions** — Blood group, allergies, ongoing medications (extracted from active prescriptions), emergency contact.

Implementation: Single FastAPI endpoint `GET /patients/{id}/passport` runs three parallel PostgreSQL queries using `asyncio.gather` for minimal response time.

Cross-Hospital Record Sharing

On booking at a new hospital, the patient sees a consent prompt to share their records. On confirmation, FastAPI creates an access grant:

```
INSERT INTO record_access_grants
  (id, patient_id, granted_to_hospital_id,
   record_types, valid_until)
VALUES
  (gen_random_uuid(), $patient_id, $hospital_id,
   ARRAY['prescriptions', 'reports'],
   now() + interval '30 days');
```

Patient can view all active grants and revoke any with one tap (`DELETE /record-access-grants/`
An RLS policy enforces that doctors at the grantee hospital can only query the patient's records while the grant is valid:

```
CREATE POLICY doctor_sees_shared_records ON prescriptions
FOR SELECT TO doctor_role
USING (
  patient_id IN (
    SELECT rag.patient_id
    FROM record_access_grants rag
    JOIN doctors d ON d.hospital_id = rag.granted_to_hospital_id
    WHERE d.user_id = auth.uid()
      AND 'prescriptions' = ANY(rag.record_types)
      AND rag.valid_until > now()
  )
```

```
  OR doctor_id = (SELECT id FROM doctors
                  WHERE user_id = auth.uid())
);
```

PostgreSQL as Full Infrastructure

Need	PostgreSQL Mechanism
Full-text search	`tsvector`, `GIN` index, `ts_rank`, `websearch_to_tsquery` on medicines and hospitals
Overbooking prevention	Atomic `UPDATE WHERE status='available'` + `UNIQUE(doctor_id, slot_start)` hard constraint
Soft-lock expiry	`pg_cron` job or APScheduler calling `UPDATE WHERE locked_until < now()` every 60 seconds
Real-time slot updates	`pg_notify` trigger on status change + `asyncpg` LISTEN in FastAPI → SSE to React
Access control	Row-Level Security (RLS) policies per role; enforced at query level independent of application code
Search result caching	`search_cache` table with `query_hash` (md5) and `expires_at`; `ON CONFLICT DO UPDATE` for upsert
Hospital search index	Materialized view combining hospital + services data with `GIN` index; refreshed on data change
Audit trail	Insert-only `appointment_status_log` table populated by trigger on every appointment status change

Search result caching with a log table (no Redis required):

```
-- Cache table
CREATE TABLE search_cache (
  query_hash TEXT PRIMARY KEY,   -- md5 of search string
  results    JSONB NOT NULL,
  cached_at  TIMESTAMPTZ DEFAULT now(),
  expires_at TIMESTAMPTZ NOT NULL
);

-- FastAPI: check cache first
SELECT results FROM search_cache
WHERE query_hash = $1 AND expires_at > now();

-- On cache miss: insert or update
INSERT INTO search_cache (query_hash, results, expires_at)
VALUES ($1, $2::jsonb, now() + interval '5 minutes')
ON CONFLICT (query_hash) DO UPDATE
```

```
SET results    = EXCLUDED.results,
    cached_at  = now(),
    expires_at = EXCLUDED.expires_at;
```

FastAPI Layer Responsibilities

FastAPI handles only what PostgreSQL cannot:

- **PDF Generation** — `weasyprint` renders the prescription HTML template to PDF; uploaded to Supabase Storage; returned as a signed, time-limited URL
- **Background Tasks** — Slot lock expiry fallback (APScheduler), waitlist cascade timer (10-minute offer window), leave notification email dispatch
- **SSE Connection Management** — Persistent `asyncpg` LISTEN connections; fan-out to all connected patients browsing a specific doctor's slots
- **Notification Dispatch** — Supabase Email and SMS triggered on booking, cancellation, prescription issuance, and waitlist offer events
- **LLM Integration** — Optional OpenRouter call to re-rank medicine suggestions when FTS returns fewer than 10 results (can be disabled for MVP demo)

Database Schema

hospitals(id, name, type, address, city, state, registration_number, admin_id, is_approved, created_at)

hospital_services(id, hospital_id [FK], service_type, service_name, department, default_duration_mins, fee, pay_at_counter, is_available)

doctors(id, user_id [FK: auth.users], hospital_id [FK], full_name, specialisation, prescription_template, verified, created_at)

patients(id, user_id [FK: auth.users], full_name, phone_number, email, dob, blood_group, known_allergies, language_preference, no_show_count, created_at)

appointment_slots(id, doctor_id [FK], slot_start, slot_end, status, locked_by, locked_until,
UNIQUE(doctor_id, slot_start))

appointments(id, slot_id [FK], patient_id [FK], doctor_id [FK], hospital_id [FK], service_id [FK], booking_type, status, notes, created_at)

slot_waitlist(id, slot_id [FK], patient_id [FK], queued_at, notified_at, offer_expires_at, status)

medicines(id, medicine_name, composition, therapeutic_class, chemical_class, uses, side_effects, substitutes, description, image_url, search_vector [generated])

prescriptions(id, appointment_id [FK], doctor_id [FK], patient_id [FK], illness_description, issued_at, pdf_url)

prescription_items(id, prescription_id [FK], medicine_id [FK], dosage, frequency, duration, doctor_comment)

patient_reports(id, patient_id [FK], hospital_id [FK], report_type, report_name, report_url, uploaded_by [FK: staff], uploaded_at)

record_access_grants(id, patient_id [FK], granted_to_hospital_id [FK], granted_to_doctor_id [FK], record_types [TEXT ARRAY], valid_until, created_at)

search_cache(query_hash, results [JSONB], cached_at, expires_at)

appointment_status_log(id, appointment_id [FK], old_status, new_status, changed_by, changed_at)

Technology Stack

Layer	Technology	Purpose
Frontend	React (Vite)	All portals; role-based routing; SSE client via EventSource
Backend API	FastAPI (Python)	REST + SSE; scheduling engine; PDF generation; LLM calls
Database	PostgreSQL / Supabase	All data; FTS; LISTEN/NOTIFY; RLS; pg_cron
Auth	Supabase Auth	Email/password, Google OAuth, Phone OTP
Email	Supabase Email	Booking confirmations, prescriptions, verification
SMS	Supabase Phone	OTP, booking SMS, waitlist offers
PDF Generation	weasyprint (Python)	Prescription PDF rendering
Storage	Supabase Storage	PDFs, reports, doctor logos
Background Jobs	APScheduler (FastAPI)	Lock expiry, waitlist cascade, leave notifications
LLM (optional)	OpenRouter	Medicine suggestion re-ranking

MVP Build Order

Phase	Timeline	Deliverable
Phase 1	Week 1	Supabase project setup; all table schemas with RLS; Supabase Auth for all three roles; FastAPI project skeleton with database connection pool

Phase	Timeline	Deliverable
Phase 2	Week 1–2	Hospital registration; doctor roster management; slot generation PostgreSQL function; calendar renders generated slots correctly
Phase 3	Week 2	Soft-lock booking flow; booking confirmation; cancellation; waitlist insert; tested under concurrent booking conditions
Phase 4	Week 2–3	Hospital FTS search; service catalogue view; booking UI with countdown timer; SSE slot updates via EventSource in React
Phase 5	Week 3	Patient profile; prescription builder in doctor interface; report upload by staff; cross-hospital access grants; health passport view
Phase 6	Week 3–4	Waitlist auto-fill via Supabase SMS; PDF prescription generation with weasyprint; hospital admin appointment dashboard; no-show marking

Three-Portal Architecture

MediNexus serves all three user types from a single React codebase using role-based routing. After login, Supabase Auth returns the user's role from the JWT claims and React redirects automatically:

- `/app/*` → **Patient Application** (hospital discovery, slot booking, health passport, record sharing management)
- `/hospital/*` → **Hospital Admin Portal** (onboarding, doctor roster, service catalogue, slot generation, appointment dashboard)
- `/doctor/*` → **Doctor Interface** (appointment queue, consultation view, prescription builder, shared patient records)